

Configure autoscaling using KEDA triggers

AI applications often experience unpredictable workloads—a surge in inference requests, batch jobs, or sudden spikes from an agent-based workflow. KEDA-based autoscaling in Azure Container Apps allows your workloads to scale to zero when idle (saving costs) and rapidly scale out when demand increases.

In this exercise, you deploy a simple mock agent API and configure autoscaling based on **HTTP concurrent requests**. You then generate concurrent load and observe how the app scales out and creates new revisions when configuration changes are applied.

Tasks performed in this exercise:

- Create Azure Container Registry and Container Apps resources
- Deploy a mock agent API container app
- Configure an HTTP concurrency scale rule using KEDA
- Generate concurrent requests to trigger scale-out and monitor replica count changes in real-time
- Configure scale rules using YAML

This exercise takes approximately **30** minutes to complete.

Important: Azure Container Registry task runs are temporarily paused from Azure free credits. This exercise requires a Pay-As-You-Go, or another paid plan.

Before you start

To complete the exercise, you need:

- An Azure subscription with the permissions to deploy the necessary Azure services. If you don't already have one, you can [sign up for one](#).
- [Visual Studio Code](#) on one of the [supported platforms](#).
- The latest version of the [Azure CLI](#).
- [Python 3.12](#) or greater.

Download project starter files and deploy Azure services

In this section you download the project starter files and use a script to deploy the necessary services to your Azure subscription. The Azure Container Registry and Container Apps environment deployment takes a few minutes to complete.

1. Open a browser and enter the following URL to download the starter file. The file will be saved in your default download location.
- Code

<https://github.com/MicrosoftLearning/mslearn-azure-ai/raw/main/downloads/python/aca-scale-python.zip>
2. Copy, or move, the file to a location in your system where you want to work on the project. Then unzip the file into a folder.
 3. Launch Visual Studio Code (VS Code) and select **File > Open Folder...** in the menu, then choose the folder containing the project files.
 4. Open the *azdeploy.py* deployment script and change the two values at the top of the script to meet your needs, then save your changes. **Note:** Do not change anything else in the script.

Code

```
"<your-resource-group-name>" # Resource Group name
"<your-azure-region>" # Azure region for the resources
```

5. In the menu bar select **Terminal > New Terminal** to open a terminal window in VS Code.
6. Run the following command to login to your Azure account. Answer the prompts to select your Azure account and subscription for the exercise.

Code	Copy
<pre>az login</pre>	

7. Run the following command to ensure you have the **containerapp** extension for Azure CLI.

Code	Copy
<pre>az extension add --name containerapp az extension add --name log-analytics</pre>	

8. Run the following commands to ensure your subscription has the necessary resource providers for the exercise.

Code	Copy
<pre>az provider register --namespace Microsoft.App az provider register --namespace Microsoft.OperationalInsights az provider register --namespace Microsoft.ContainerRegistry</pre>	

Create resources in Azure

In this section you run the deployment script to deploy the necessary services to your Azure subscription.

1. Make sure you are in the root directory of the project and run the following command in the terminal to launch the deployment script. The script deploys ACR, a Container Apps environment, and a Container App with ingress enabled. It also creates the environment variable files you use throughout the exercise.

Code	Copy
<pre>python azdeploy.py</pre>	

2. When the script is running, enter **1** to launch **Create Azure Container Registry and build container image**.
3. When the previous operation is finished, enter **2** to launch **Create Container Apps environment**.
4. When the previous operation is finished, enter **3** to launch **Create Container App**.

Note: A file containing environment variables is created after the container app is created. You use these variables throughout the exercise.

5. When the deployment completes, enter **5** to exit the deployment script.
6. Run the appropriate command to load the environment variables into your terminal session from the file created in a previous step.

Bash

Code	Copy
<pre>source .env</pre>	

PowerShell

Code	Copy
<pre>. .\env.ps1</pre>	

7. Verify the app endpoint is available.

Bash

Code	Copy
<pre>curl -sS "\$CONTAINER_APP_URL/" head</pre>	

PowerShell

Code	Copy
<pre>Invoke-RestMethod "\$env:CONTAINER_APP_URL/"</pre>	

Note: Keep the terminal open. If you close it and create a new terminal, you might need to run the command to create the environment variable again.

Configure autoscaling

In this section you configure an HTTP scale rule that triggers scaling based on **concurrent requests**. This is a useful proxy for "agent requests in progress" without adding any other Azure services.

Note: Applying configuration updates (including scaling changes) creates a **new revision**.

1. Run the following command to update the container app with an HTTP scale rule. This rule monitors concurrent in-flight requests and scales the app when demand increases.

Bash

CodeCopy

```
az containerapp update \  
  --name $CONTAINER_APP_NAME \  
  --resource-group $RESOURCE_GROUP \  
  --min-replicas 0 \  
  --max-replicas 10 \  
  --scale-rule-name http-scaling \  
  --scale-rule-type http \  
  --scale-rule-http-concurrency 10
```

PowerShell

CodeCopy

```
az containerapp update `\  
  --name $env:CONTAINER_APP_NAME `\  
  --resource-group $env:RESOURCE_GROUP `\  
  --min-replicas 0 `\  
  --max-replicas 10 `\  
  --scale-rule-name http-scaling `\  
  --scale-rule-type http `\  
  --scale-rule-http-concurrency 10
```

2. Run the following command to verify the scale rule is configured. Look for the **http-scaling** rule in the output with **minReplicas** set to **0** and **maxReplicas** set to **10**.

Bash

CodeCopy

```
az containerapp show \  
  --name $CONTAINER_APP_NAME \  
  --resource-group $RESOURCE_GROUP \  
  --query "properties.template.scale"
```

PowerShell

CodeCopy

```
az containerapp show `\  
  --name $env:CONTAINER_APP_NAME `\  
  --resource-group $env:RESOURCE_GROUP `\  
  --query "properties.template.scale"
```

Generate load and observe scaling

In this section you run a local Flask dashboard that can both generate concurrent requests and show Container App revisions/replicas.

1. Run the following command to navigate to the *client* directory.

CodeCopy

cd client

2. Run the following command to create a virtual environment for the client app. Depending on your environment the command might be **python** or **python3**.

CodeCopy

python -m venv .venv

3. Run the following command to activate the Python environment. **Note:** On Linux/macOS, use the Bash command. On Windows, use the PowerShell command. If using Git Bash on Windows, use **source .venv/Scripts/activate**.

Bash

CodeCopy

source .venv/bin/activate

PowerShell

CodeCopy

.\.venv\Scripts\Activate.ps1

4. Run the following command to install the dependencies for the client app.

CodeCopy

pip install -r requirements.txt

5. Run the following command to start the dashboard.

CodeCopy

python app.py

6. Open a browser and navigate to the following URL: **http://127.0.0.1:5000**.
7. In the left pane of the app select **Refresh Revisions & Replicas**. In the top right of the app you should see **1**, or **0** replicas running.

When you deployed the app it defaulted to **1** running replica. You applied KEDA scale rule in a previous step and scaling down to zero may take an additional **~5 minutes** after the workload becomes idle because of the default **300-second (5-minute)** cool-down period.

8. In the **Load Generator** section, select **Start** to being sending data to the container app.
9. Select **Refresh Revisions & Replicas** every 5-10 seconds and you should see the number of replicas increase. You can run the **Load Generator** again after it stops to increase the traffic and increase replica count.

When you're finished close the browser window and enter **Ctrl+c** in the terminal to end the client app.

Configure scale rules using YAML

In this section you configure autoscaling by editing the Container App YAML. This is a repeatable way to manage scale rules, and it becomes essential when you have multiple rules.

1. Run the following command to export the app configuration to a YAML file.

Bash

CodeCopy

az containerapp show \
 --name \$CONTAINER_APP_NAME \
 --resource-group \$RESOURCE_GROUP \
 --output yaml > app-config.yaml

PowerShell

CodeCopy

```
az containerapp show `
  --name $env:CONTAINER_APP_NAME `
  --resource-group $env:RESOURCE_GROUP `
  --output yaml > app-config.yaml
```

2. Open the *app-config.yaml* file in VS Code. Find the **scale** section under **properties > template**. Modify the scaling configuration to reduce the **cooldownPeriod** to **200** seconds (faster scale-down), set **maxReplicas** to **5**, and set **minReplicas** to **1** so the app always has at least one replica running. The **scale** section should look similar to the following example.

CodeCopy

```
scale:
  cooldownPeriod: 200
  maxReplicas: 5
  minReplicas: 1
  pollingInterval: 30
```

3. Save the file and run the following command to apply the updated configuration.

Bash

CodeCopy

```
az containerapp update \
  --name $CONTAINER_APP_NAME \
  --resource-group $RESOURCE_GROUP \
  --yaml app-config.yaml
```

PowerShell

CodeCopy

```
az containerapp update `
  --name $env:CONTAINER_APP_NAME `
  --resource-group $env:RESOURCE_GROUP `
  --yaml app-config.yaml
```

4. Run the following command to verify the changes you just implemented.

Bash

CodeCopy

```
az containerapp show \
  --name $CONTAINER_APP_NAME \
  --resource-group $RESOURCE_GROUP \
  --query "properties.template.scale"
```

PowerShell

CodeCopy

```
az containerapp show `
  --name $env:CONTAINER_APP_NAME `
  --resource-group $env:RESOURCE_GROUP `
  --query "properties.template.scale"
```

Clean up resources

Now that you finished the exercise, you should delete the cloud resources you created to avoid unnecessary resource usage.

1. Run the following command in the VS Code terminal to delete the resource group, and all resources in the group. Replace **<rg-name>** with the name you choose earlier in the exercise. The command will launch a background task in Azure to delete the resource group.

Code	 Copy
<pre>az group delete --name <rg-name> --no-wait --yes</pre>	

CAUTION: Deleting a resource group deletes all resources contained within it. If you chose an existing resource group for this exercise, any existing resources outside the scope of this exercise will also be deleted.

Troubleshooting

If you encounter issues during this exercise, try these steps:

App not scaling out under load

- Verify the HTTP scale rule is configured: **az containerapp show --query "properties.template.scale"**
- Ensure you're generating concurrent requests (use the dashboard with delayMs > 0)
- Increase **delayMs** (500-1500ms) so requests overlap and concurrency accumulates
- Check system logs for scaling events: **az containerapp logs show --type system --tail 50**

Dashboard won't start or can't list revisions/replicas

- Ensure Python virtual environment is activated (you should see **(.venv)** in your terminal prompt)
- Ensure dependencies are installed: **pip install -r client/requirements.txt**
- Ensure Azure CLI is installed and you ran **az login**
- Ensure the **containerapp** extension is installed: **az extension add --name containerapp**
- Ensure **.env** is loaded and contains **RESOURCE_GROUP** and **CONTAINER_APP_NAME**

Python venv activation issues

- On Linux/macOS, use: **source client/.venv/bin/activate**
- On Windows PowerShell, use: **.\client.venv\Scripts\Activate.ps1**
- If **activate** script is missing, reinstall **python3-venv** package and recreate the venv

YAML update fails

- Ensure the YAML file syntax is valid (check indentation)
- Some read-only properties like **id**, **systemData**, and **type** may cause errors; remove them if needed
- Verify the scale section follows the correct structure under **properties > template > scale**